

Smart WMS / DSS Documentation

This document explains how the system works, its features, navigation, login flow, and how to test each major function.

1) What This System Is

Smart WMS (Warehouse Management System) is a desktop application built with PySide6 and SQLite.

It helps warehouse teams monitor inventory health, execute stock operations, and receive decision support recommendations.

Core capabilities:

- Inventory and stock management (products, locations, suppliers, POs, movements)
- Decision support engine (restock urgency, alerts, priority actions)
- Role-based access (Admin vs Viewer)
- Barcode-assisted workflows
- Reports and analytics

2) Tech Stack

- **Language:** Python
- **UI:** PySide6 (Qt)
- **Database:** SQLite (`dss\_database.db`)
- **Excel import/export:** `openpyxl`
- **Packaging:** PyInstaller

Key files:

- `main.py` - app entry point and main window
- `services/data\_service.py` - database schema + business data operations
- `logic/decision\_engine.py` - scoring and recommendations
- `ui/dashboard.py`, `ui/management.py`, `ui/reports.py`, `ui/settings.py` - primary tabs

3) Installation and Running

Prerequisites

- Python 3.10+ recommended

Install dependencies

```
```bash
pip install -r requirements.txt
```
```

Run the app

```
```bash
```

```
python main.py
```

```
...
```

### ***Build executable***

```
```bash
```

```
pyinstaller dss.spec
```

```
...
```

or on PowerShell:

```
```powershell
```

```
.\build.ps1
```

```
...
```

### ***Generate sample/template Excel files***

```
```bash
```

```
python scripts/data/generate_warehouse_excels.py
```

```
...
```

4) Login and Accounts

On startup, the app opens a login dialog.

Default seeded accounts:

- **Admin:** `admin` / `admin123`
- **Viewer:** `viewer` / `viewer123`

Role behavior:

- **Admin:** opens the **Admin Window** (`MainWindow`) with full access: Dashboard, Inventory, Reports, Settings.
- **Viewer:** opens a separate **Viewer Window** (`ViewerWindow`) with Dashboard, Inventory (read-only), Reports only.

Window behavior:

- `Change Account` closes the current window and opens a new one for the selected account role.
- Admin and Viewer use separate window classes and navigation layouts.

Where implemented:

- Login dialog: `LoginDialog` in `main.py`
- Authentication: `DataService.authenticate\_user()` in `services/data\_service.py`

5) How Navigation Works

Admin Window

The left sidebar has four main tabs:

1. **Dashboard**

2. **Inventory**
3. **Reports**
4. **Settings**

There is also:

- **Change Account** button (switch user without restarting)
- **Barcode Scanner toggle** (Active / Disabled)

Viewer Window

Viewer sidebar includes only:

1. **Dashboard**
2. **Inventory (Read-Only)**
3. **Reports**

Viewer window does not include `Settings`.

Main navigation logic is in `MainWindow` and `ViewerWindow` (`main.py`).

6) Tab-by-Tab Feature Guide

Dashboard

Purpose:

- Quick operational overview and priority decision queue.

What you can do:

- View high/medium/low recommendations.
- See active alerts.
- Trigger action workflows from recommendations.
- Generate purchase planning suggestions.

Common use:

- Start your day here to see what needs immediate attention.

Inventory

Purpose:

- Central workspace for all warehouse master and transaction data.

Sub-tabs and purpose:

- **Products**
 - Add/edit/delete products (admin)
 - Set reorder points, supplier, default location (admin)
 - Barcode metadata lookup/autofill
 - Import/export product workbook (admin)

- **Locations**
 - Manage warehouse location map (admin)
- **Stock Levels**
 - Execute stock movements (admin):
 - `IN`
 - `OUT`
 - `ADJUST`
 - `TRANSFER`
- **Movements**
 - View movement history/audit records
- **Suppliers**
 - Add and maintain supplier records (admin)
 - Activate/deactivate supplier usage (admin)
- **Purchase Orders**
 - Create/manage PO drafts and lines (admin)
 - Receive PO and update stock (admin)
- **Cycle Count**
 - Compare counted vs system quantity
 - Apply variance corrections (admin)
- **Users** (admin-focused)
 - Create users
 - Reset passwords
 - Activate/deactivate accounts

Viewer UI behavior in Inventory:

- Edit-only controls are hidden (not just disabled), including Add/Import/Export and mutation action buttons.
- Viewer can still refresh and inspect data tables.

Reports

Purpose:

- Performance and stock-health analytics.

What you can see:

- KPI cards (SKU count, inventory value, etc.)
- Insights generated by decision engine
- ABC analysis
- Category and location utilization breakdowns

Settings

Purpose:

- Control recommendation behavior and thresholds.

What you can configure:

- Decision weights (urgency/frequency/impact/recency)
- Priority thresholds
- Popup controls (cooldown/confidence/enable)

Notes:

- Settings changes are persisted and reloaded into runtime logic.

7) How the System Works Internally (High Level)

1. User logs in (`main.py`).
2. Data service connects to SQLite and ensures schema exists (`DataService`).
3. UI pages load from database state.
4. Decision engine computes recommendation scores from inventory context.
5. Dashboard/Reports display recommendations and insights.
6. User actions (movements, PO receive, cycle count) update database and refresh analytics.

8) Barcode Workflow

Two common paths:

- **Known barcode**
 - Scan -> product found -> action dialog appears -> stock action can be applied.
- **Unknown barcode**
 - Scan -> unknown dialog -> create product flow -> optional stock update.

Relevant files:

- `logic/barcode\_scanner.py`
- `ui/scan\_dialog.py`
- `ui/management.py` (`AddProductDialog` barcode autofill)

9) Database and Data Model Summary

Major tables include:

- `products`
- `locations`
- `stock\_levels`
- `stock\_movements`
- `suppliers`

- `purchase\_orders`
- `purchase\_order\_lines`
- `alerts`
- `recommendations`
- `user\_behavior`
- `barcode\_scans`
- `barcode\_profiles`
- `users`

Schema creation is defined in `DataService.\_initialize\_database()`.

10) Feature Testing Guide (Manual QA)

Use this checklist to verify key functions after changes.

A. Login / Access Control

- Login as `viewer` and confirm restricted behavior.
- Login as `admin` and confirm full control.
- Test `Change Account` from sidebar.
- Confirm `Change Account` closes current window and opens the correct role window.
- Confirm Viewer does not see Settings tab.
- Confirm Viewer Inventory hides edit/import/export/action buttons.

B. Products and Master Data

- Add new product with initial stock.
- Add new location and supplier.
- Verify entries appear correctly in tables.

C. Stock Operations

- Post one `IN`, one `OUT`, one `ADJUST`, one `TRANSFER`.
- Confirm quantity updates and movement logs are correct.

D. Purchase Orders

- Create PO (or generate via Dashboard plan).
- Receive PO.
- Verify stock increments and movement entries.

E. Cycle Count

- Open cycle count tab.
- Enter counted quantity different from system quantity.
- Apply and verify variance correction.

F. Reports

- Open Reports tab and validate KPIs populate.
- Confirm insights and analysis sections refresh after inventory changes.

G. Barcode

- Scan known barcode and complete a stock action.
- Scan unknown barcode and create product from scan flow.

H. Settings

- Change one weight/threshold.
- Save.
- Verify recommendations change behavior after refresh.

I. End-to-End Smoke Test

- Create product -> move stock -> generate recommendation -> execute action -> verify report impact.

11) Helpful Scripts

- ``scripts/tests/test_warehouse.py`` - broad warehouse logic check
- ``scripts/tests/test_barcode.py`` - barcode behavior checks
- ``scripts/build/create_icon.py`` - generate app icon if needed

Build helper:

- ``build.ps1`` always rebuilds from current source and outputs ``dist/SmartDSS.exe``.

12) Configuration Notes

Configuration is stored in:

- ``config/__init__.py``

Important values:

- DB path/name
- app size and metadata
- recommendation weights
- threshold values
- popup controls

When bundled, database path may use ``%APPDATA%`` for writable storage.